

Cowrie SSH Honeypot Deployment in Proxmox

Project Overview

As part of my cybersecurity homelab, I deployed a Cowrie SSH honeypot inside a Proxmox virtualized environment. The purpose of this project was to gain hands-on experience with Linux administration, virtualization, honeypots, and attack monitoring. Rather than exposing the system directly to the internet, I first built and tested everything within my isolated lab environment using Kali Linux as the attacking machine.

This project allowed me to better understand how attackers interact with exposed SSH services and how security professionals can collect and analyze attack data using honeypots.

Environment

Hardware

- HP EliteDesk 800 G6 Mini
- Intel Core i5-10600T (6 Cores / 12 Threads)
- 16 GB DDR4 RAM
- 512 GB NVMe SSD

Virtualization Platform

- Proxmox VE

Virtual Machines

- Ubuntu Server 24.04 LTS (Cowrie Honeypot)
- Kali Linux (Attack Machine)

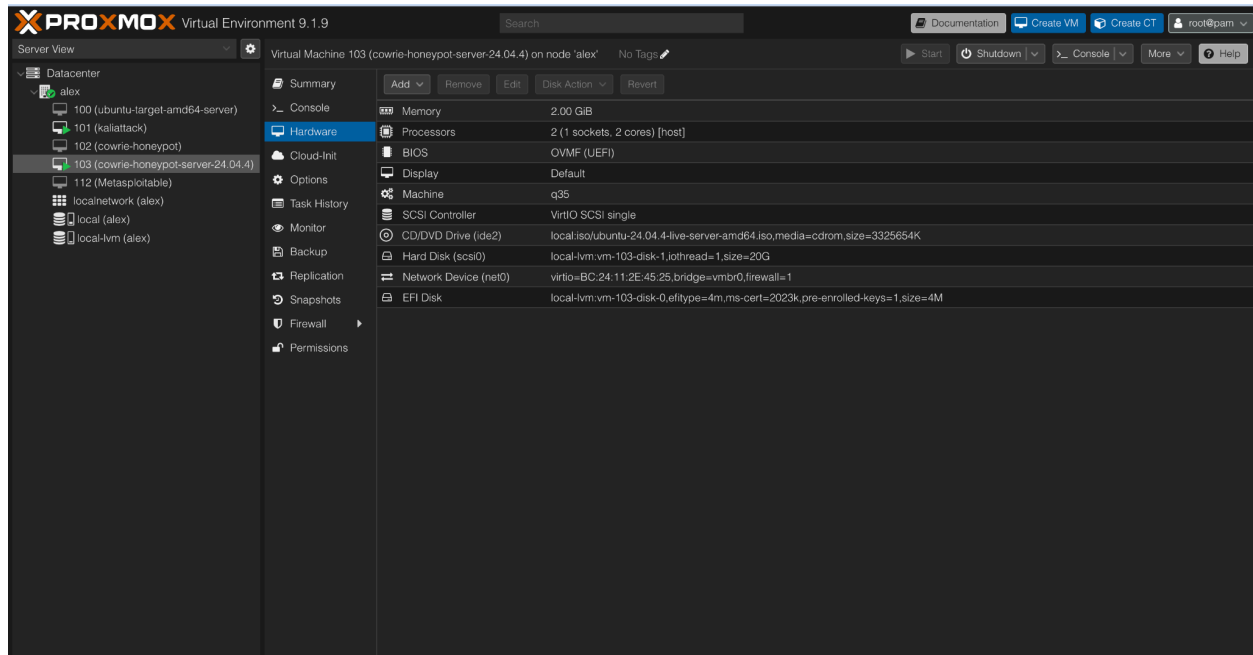
Project Goals

The primary goals of this project were:

- Deploy a functional SSH honeypot
- Simulate attacks from a separate attacker machine
- Capture login attempts and attacker activity

- Gain experience with Linux server administration
- Learn how honeypots can be used for threat monitoring and research

Building the Honeypot



I began by creating a dedicated Ubuntu Server virtual machine in Proxmox. The VM was configured with 2 CPU cores, 2 GB of RAM, and a 20 GB virtual disk. During installation, OpenSSH Server was enabled so the system could be managed remotely.

After updating the operating system and installing the required dependencies, I created a dedicated user account for Cowrie and cloned the official Cowrie repository from GitHub. The installation process also required creating a Python virtual environment and installing all project dependencies.

Challenges and Troubleshooting

One of the most valuable parts of this project was troubleshooting several unexpected issues during deployment.

Initially, I attempted to deploy Cowrie on Ubuntu 26.04. While the operating system installed successfully, many of the online tutorials I followed referenced an older version of Cowrie. As a result, several expected files and startup scripts were missing.

For example, many guides referenced configuration files located in the `etc` directory and a startup script called `bin/cowrie`. After investigating the repository structure, I discovered that newer versions of Cowrie had reorganized these files into different locations. This caused

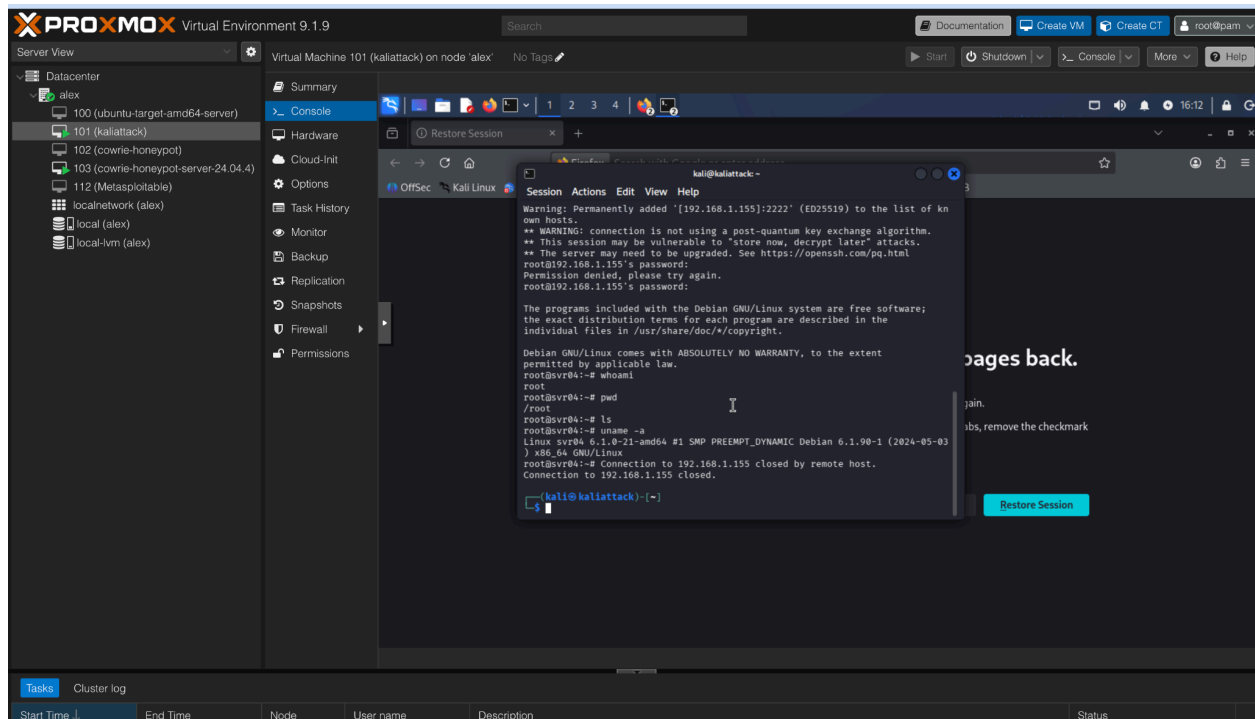
several installation attempts to fail until I manually located the configuration files and adjusted the setup process.

I also encountered issues with Python module recognition inside the virtual environment. After additional troubleshooting, I determined that Cowrie needed to be installed into the virtual environment using editable mode before it could be started correctly.

Rather than continuing to troubleshoot on Ubuntu 26.04, I decided to rebuild the honeypot using Ubuntu Server 24.04 LTS. This decision significantly improved compatibility with available documentation and allowed the deployment to proceed successfully.

Throughout the process I gained practical experience reading Linux error messages, navigating file structures, managing Python virtual environments, and diagnosing installation problems.

Attack Simulation

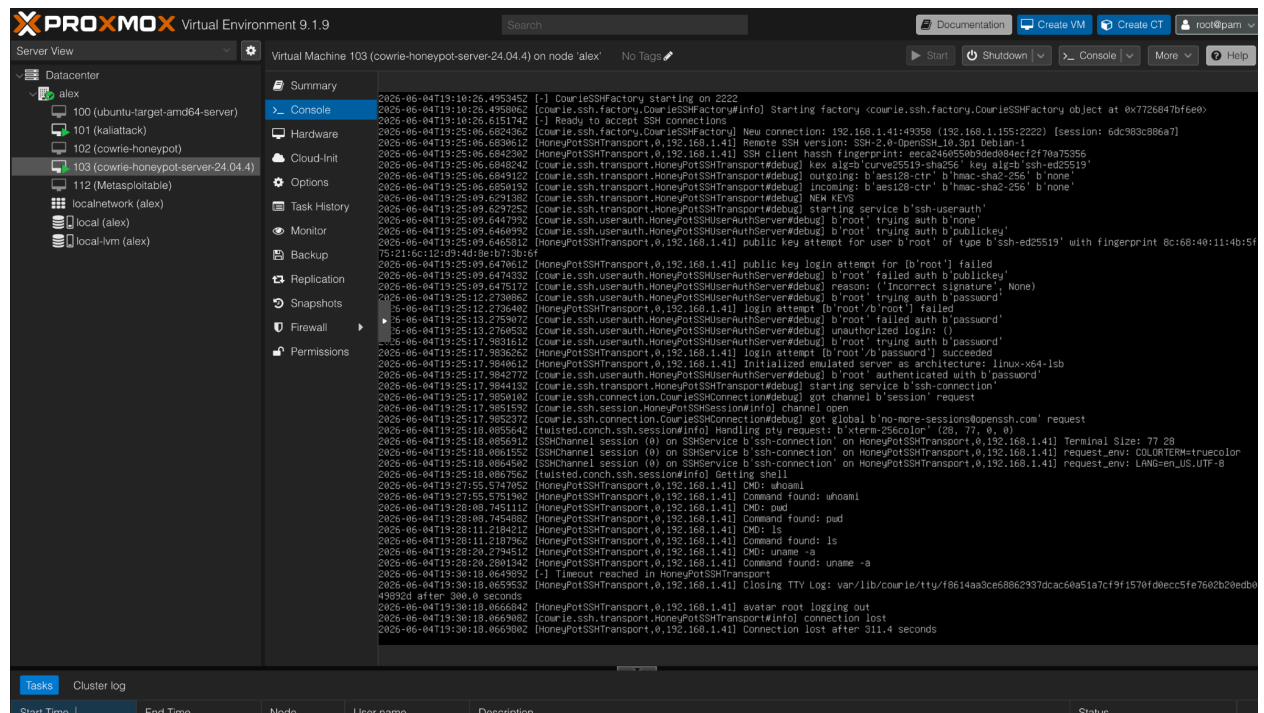


Once the honeypot was operational, I used my Kali Linux VM to simulate SSH attacks against the Cowrie server.

During testing, I observed that some login attempts were intentionally rejected while commonly used passwords were accepted. This behavior demonstrated how Cowrie emulates a vulnerable system by presenting attackers with a realistic but completely fake shell environment.

From the attacker's perspective, it appeared as though access had been successfully obtained. In reality, every action was being monitored and recorded by the honeypot.

Monitoring and Log Analysis



To verify that the honeypot was functioning correctly, I monitored Cowrie's log files in real time.

Using the log output, I was able to observe:

- Incoming SSH connections
- Usernames entered by attackers
- Password attempts
- Session activity
- Commands executed inside the fake shell

Watching these events appear live while attacking from Kali provided valuable insight into how honeypots collect intelligence and document attacker behavior.

Skills Developed

This project helped strengthen my skills in:

- Proxmox virtualization
- Linux server administration
- SSH services and authentication
- Python virtual environments
- Git and GitHub repositories

- Cybersecurity monitoring
- Log analysis
- Honeypot deployment and management
- Troubleshooting and problem solving

Future Improvements

This honeypot currently operates within a controlled lab environment. Future plans include:

- Integrating Cowrie logs into Wazuh for centralized monitoring
- Deploying pfSense to create a segmented security lab
- Building dashboards for attack visualization
- Safely exposing the honeypot to the internet for real-world data collection
- Expanding the environment with additional security tools and monitoring capabilities

Conclusion

This project successfully demonstrated the deployment of a functional SSH honeypot within a Proxmox-based homelab. Beyond simply installing the software, the project provided valuable experience troubleshooting real-world deployment challenges and understanding how attackers interact with exposed systems. The experience strengthened both my Linux administration skills and my understanding of defensive cybersecurity techniques, while creating a foundation for future projects involving SIEM platforms, firewalls, and threat monitoring.